

MAE 3780 Robot Project Final Report

Group 42 - Ashley Padres (ap974), Anna Schaldenbrand (ams796), Hannah Sherman (hms256)

1. Robot Design and Strategy Overview.

The primary goal of this design is maximizing passive block collection, as opposed to defensive or offensive strategies. The main goal in our block collecting strategy was to quickly collect more than half of the blocks and leave the board.

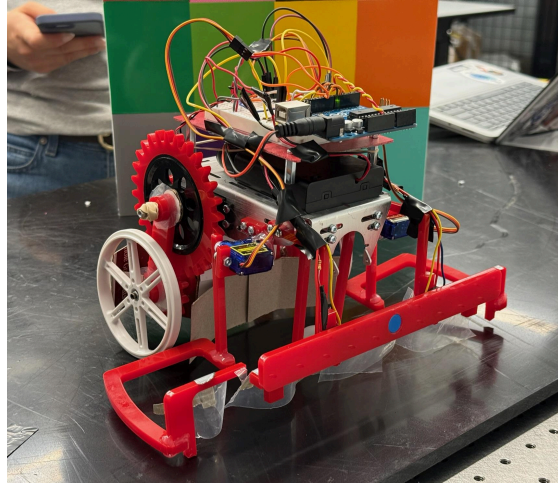


Figure 1. Final robot – arms deployed

Mechanical Design

- Gear train: since the main goal of our robot was to get to the center of the board as fast as possible, we used the mechanical advantage of a gear reduction to maximize speed. We utilized the given small wheels as large gears by laser cutting acrylic adapters that made them into large wheels. They were attached to a gear train that drove our large wheels faster.
- Deployable arms + collector screen: In order to maximize the collection area, acrylic deployable arms were used in the front of the robot that made use of the 12" diameter bounding cylinder allowed after deployment. A one-way collector screen fashioned from a plastic sheet collects blocks in the front, with pieces attached to both the push bar and the deployable bars.
- Push bar: A laser cut acrylic bar is mounted statically to the front of the car via two buttons which are triggered when the robot runs into another robot.

Electrical Design

- Parallel button: the two buttons are wired in parallel so that a high signal from either will trigger the interrupt to change paths. The buttons are wired with pull-down resistors so that a high signal will trigger when either button is pressed. This will trigger the alternative path for the robot to move backwards.

Software Design

- At the beginning of the deployment sequence, both arms drop down via two servos controlled by a 50 Hz PWM signal from Timer1. Arm 1 is connected to servo 1 which is a positional motor set to a fixed angle. Arm 2 is connected to servo 2 which is a

continuous rotation servo whose speed and direction are set by a pulse width. The pulse widths were set using Output Compare Registers 1 and 2 (OCR1 and OCR2).

- Hard coded path: During competition, barring no interruptions from other robots, our robot drives forward to the middle and turns right, ideally collecting just over half of the blocks on the right side of the board. It then continues and drives off the board and stops, since there are no out-of-bounds.
- Button interrupt: Additionally, if the robot hits the other robot, our robot has a “push-bar” (consisting of two buttons) attached to the front that triggers an external interrupt. This interrupt causes the robot to reverse off the board and stop, collecting extra blocks as it moves backward as well.

2. Design Process Reflection.

Throughout the course of this project, our design evolved rapidly and drastically in direct response to our initial testing results. While our main objectives of 1.) be ‘fast’, 2.) collect and retain blocks, and 3.) avoid contact with other robots, remained throughout our design iterations, the method to achieving those goals changed.

As mentioned in the strategy overview, one of the primary design objectives for our robot was to have a robot that was fast, as with a fast robot, we could get to the blocks before our potential opponent. Our initial strategy to achieve faster speeds was to combine the use of larger wheels with an amplified voltage input to the servos, via the use of an operational amplifier (op-amp). While this idea seemed initially promising, there were a few difficulties that we ran into along the way. Primarily, we found that when trying to connect the output from the op-amp directly to the h-bridge input, the op-amp output voltage would drop in an unexpected way, not providing an amplified voltage as expected. However, there was a bigger problem with our use of an op amp that we had initially missed– the use of additional 9V batteries (such as those needed to power the op-amp) was not allowed per the rules.

As a result, we decided to pursue a gear train. However, since we decided to pivot our design quite late, we didn’t have the option to purchase servo gears from a company like Polulu, so instead we designed a custom gear train using gear designs from McMaster and created a gear assembly that could be cut from acrylic and could interface with the spare wheels we had to attach the gears to the servo. This design change required quite a lot of debugging and testing, and the final version for competition involved us removing the rubber from the wheels to reduce a stall failure mode we were seeing due to our high RPM, low torque gear train.

The second critical part of our design was the block retaining mechanism, which was a sheet of plastic designed to fold in one direction, when the robot was driving over blocks, and was designed to not fold in the opposite direction, to retain blocks when driving backward. This part of the design didn’t evolve much, as initial testing with popsicle sticks and cardboard demonstrated that our design would be sufficient.

The last important part of our design was the button feature at the front of the robot. This feature was designed such that if we run into another robot or obstacle, the robot would simply

reverse, while retaining the blocks it collected, to avoid the other robot. This feature took a bit of iterative development. Initially, the button feature was designed as a piece of acrylic attached directly to one button. However, initial testing demonstrated that this button was not stiff enough, and if the acrylic piece was pushed off-axis, the button wouldn't be triggered. As a result, we improvised and switched to using two buttons attached to the one piece of acrylic, to increase the stability of the component and to ensure the button would be triggered upon contact. Due to the fact that we wanted our "drive backward" interrupt to be triggered when either button was pressed, we tried to implement the use of an Or Gate component. However, following some issues wiring the component, we decided to get around it by wiring the button outputs in parallel.

3. Competition Analysis.

The robot competed in 6 rounds and ended with a 3-2-1 score for W-L-T. We noticed that the 2 rounds that ended in losses happened because the button-activation that caused the robot to reverse, ended in us losing a majority of the blocks we had already collected. This is largely because the plastic one-way-gate we implemented did not work as well as when we had tested it in the lab, so when we reversed over the edge of the board, we ended up losing a significant number of blocks. To fix this problem, we un-plugged the wires that electrically connected the button to the robot to avoid triggering the robot reversing direction during the competition. This helped us secure 3 wins and 1 tie in the following four rounds.

In our best round, we collected 12 blocks from the board, which was made possible by the extra area we achieved by using our micro-servos to deploy additional arms and the speed gained by our geartrain. The deployable arms worked as expected during every round of competition, which helped us increase our robot's perimeter and the gear-train was mechanically successful so we were able to increase our speed in collecting blocks before other robots got to the center of the board. Although we did not advance to the round robin, we are pleased with the mechanical and electrical success of our robot and that we were able to collect a significant number of blocks in 4 out of 6 rounds.

4. Conclusions.

If we could have redone this project again, my team has realized that our design likely could have been more streamlined if we were able to test out these major design decisions earlier in the project, so that time-wise, we would have been less constrained in what components we could purchase and use. Also, we would have been able to do more edge-case testing, as, during the competition, we realized that our block retaining mechanism was ineffective when driving backward off the map, which led us to intentionally disconnecting a wire to prevent us from driving backward off the map mid-competition. In the end, the best piece of advice we could give is to ask your instructors what is reasonable to do with the limited time; while the milestones give guidance on progress, if a team really wants to pursue a unique or innovative idea, progress needs to be made toward block collecting and color sensing well in advance to the milestones, to avoid last minute changes as we did.

5. Appendix A: Bill of Materials

Item	Source	Use	QTY	Cost/Unit	Cost	Total Cost
Acrylic Plate	<u>RPL</u>	Arms, Geartrain, Button	1	\$5.00	\$5.00	\$39.87
Laser Cut/Inch	<u>RPL</u>	Arms, Geartrain, Button	98	\$0.05	\$4.90	
Acrylic Pieces	<u>RPL</u>	Arms, Geartrain, Button	1	\$0.50	\$0.50	
Pololu Wheels	https://www.pololu.com/product/4939	Driving	2	\$2.00	\$4.00	
Epoxy	https://www.mcmaster.com/7541A51/	Acrylic Adhesion	1	\$6.71	\$6.71	
Axel	Stand-off from 3780 Lab	Geartrain			\$0.00	
Timing belt 1	https://www.mcmaster.com/7887K2/	Geartrain (unused in final design)	2	\$4.60	\$9.20	
Timing belt 2	https://www.mcmaster.com/7887K23/	Geartrain (unused in final design)	2	\$4.78	\$9.56	
Bag	From McMaster Parts (Lines 6, 8, 9)	1-way gate for block collection			\$0.00	

6. Appendix B: Circuit Diagram

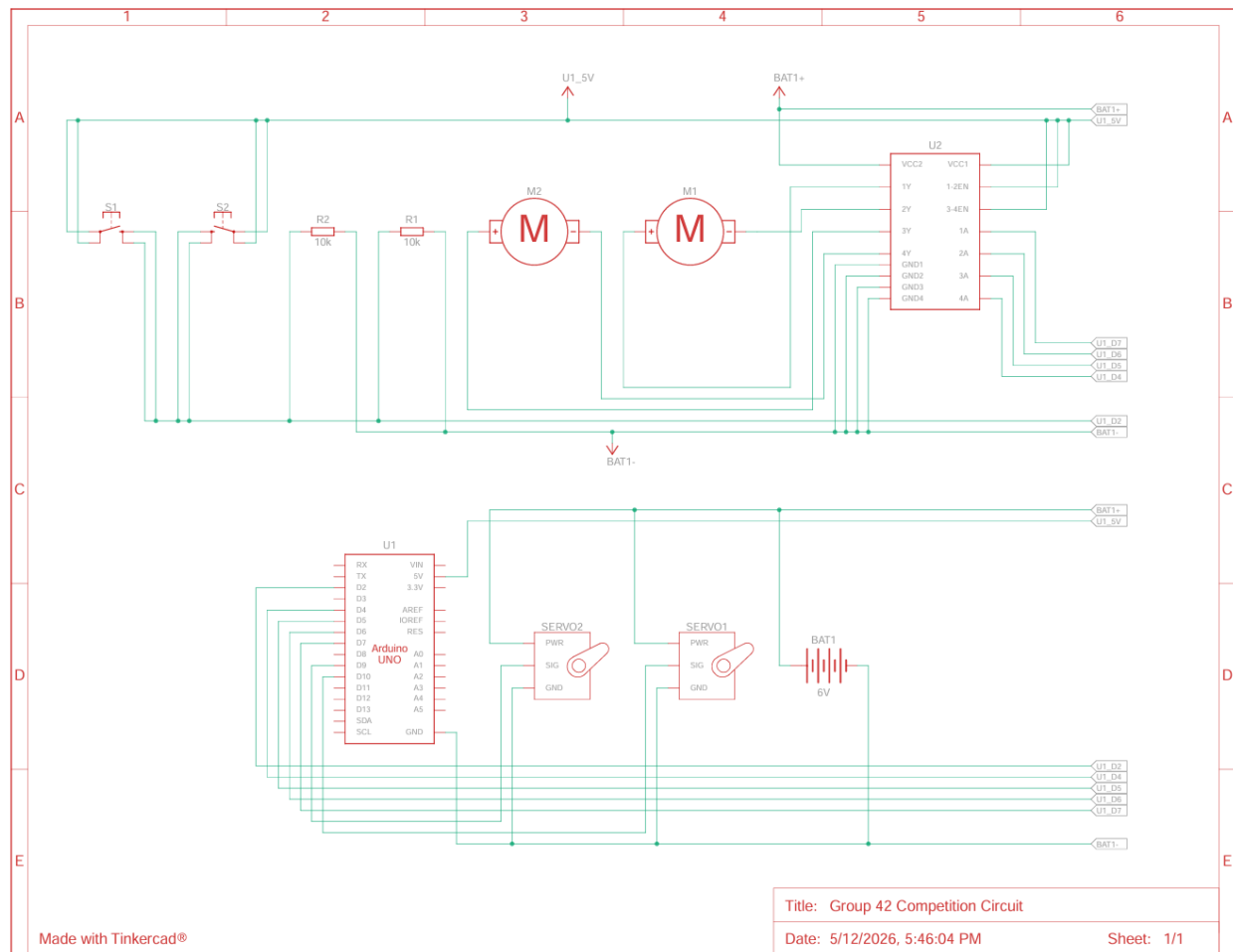


Figure 2. Final competition circuit diagram, created using Tinkercad circuit software.

7. Appendix C: CAD Files and Drawings

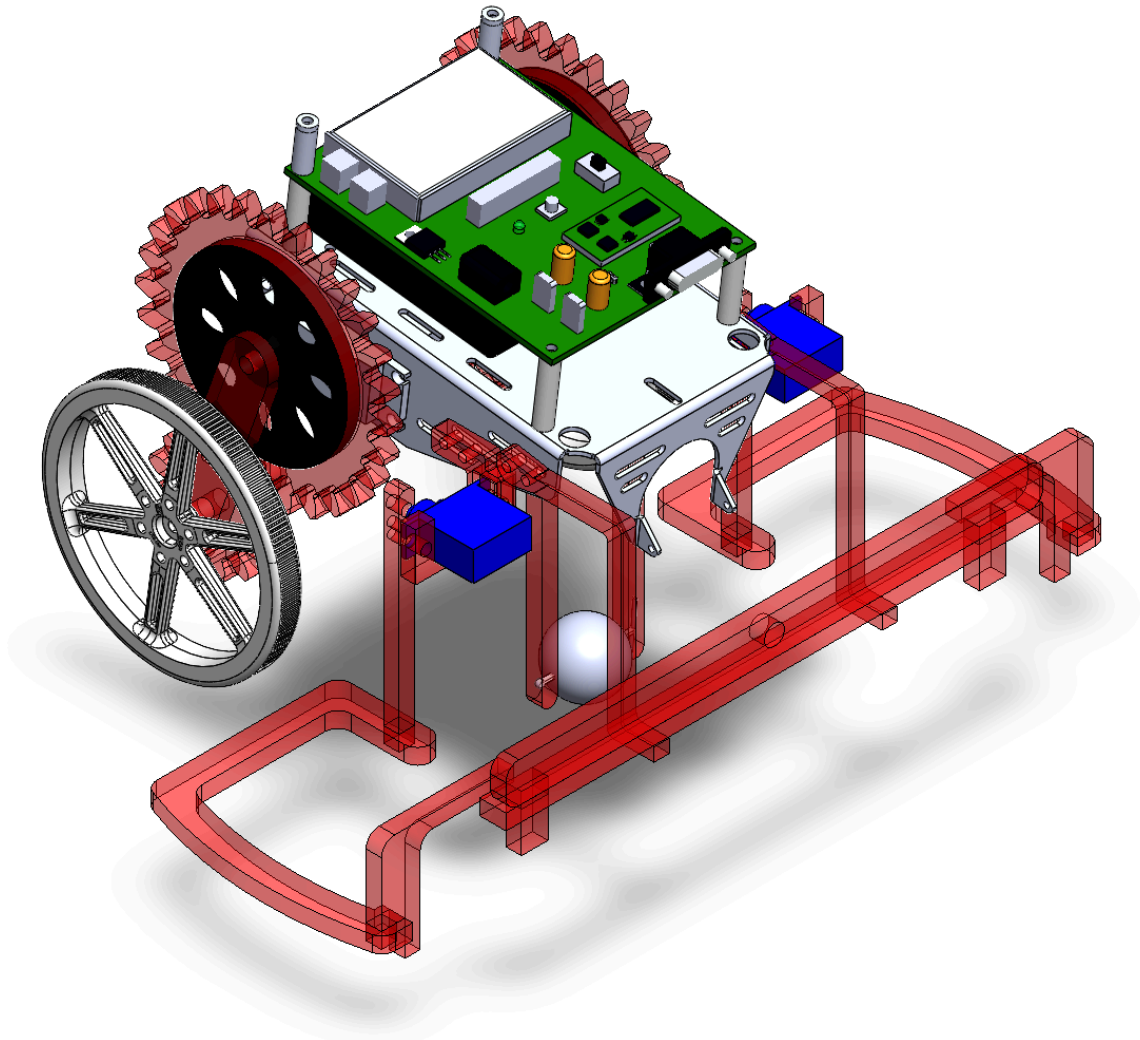


Figure 3: Isometric view of robot CAD assembly

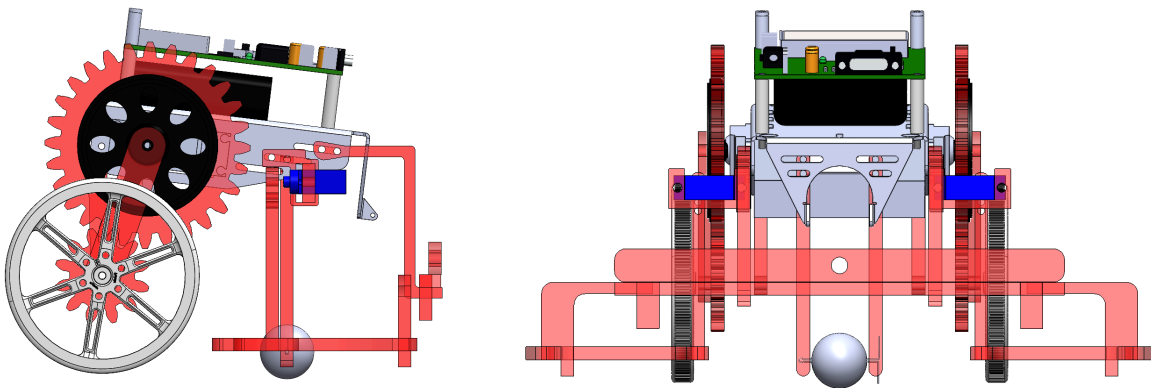


Figure 4: Side view (left) and front view (right) of robot CAD assembly

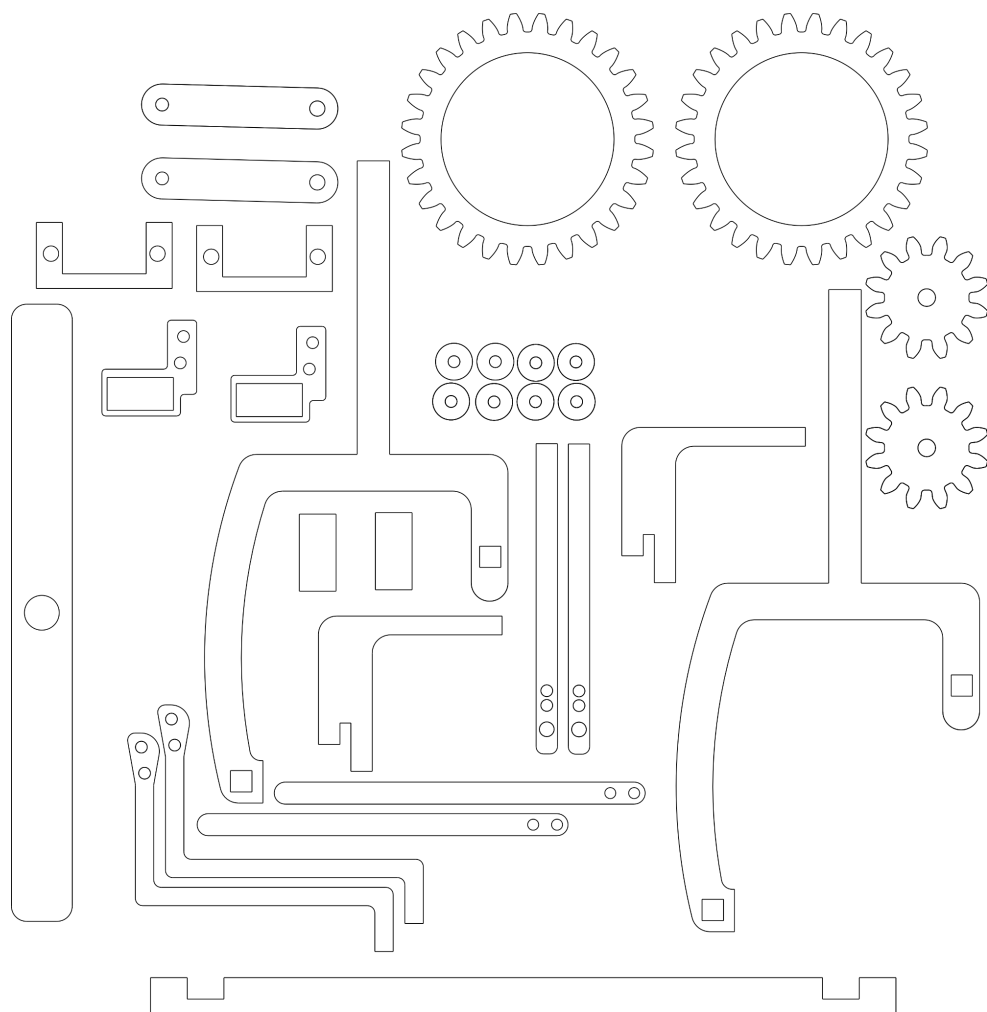


Figure 5. DXF files used to laser cut from acrylic

8. Appendix D: Flowchart

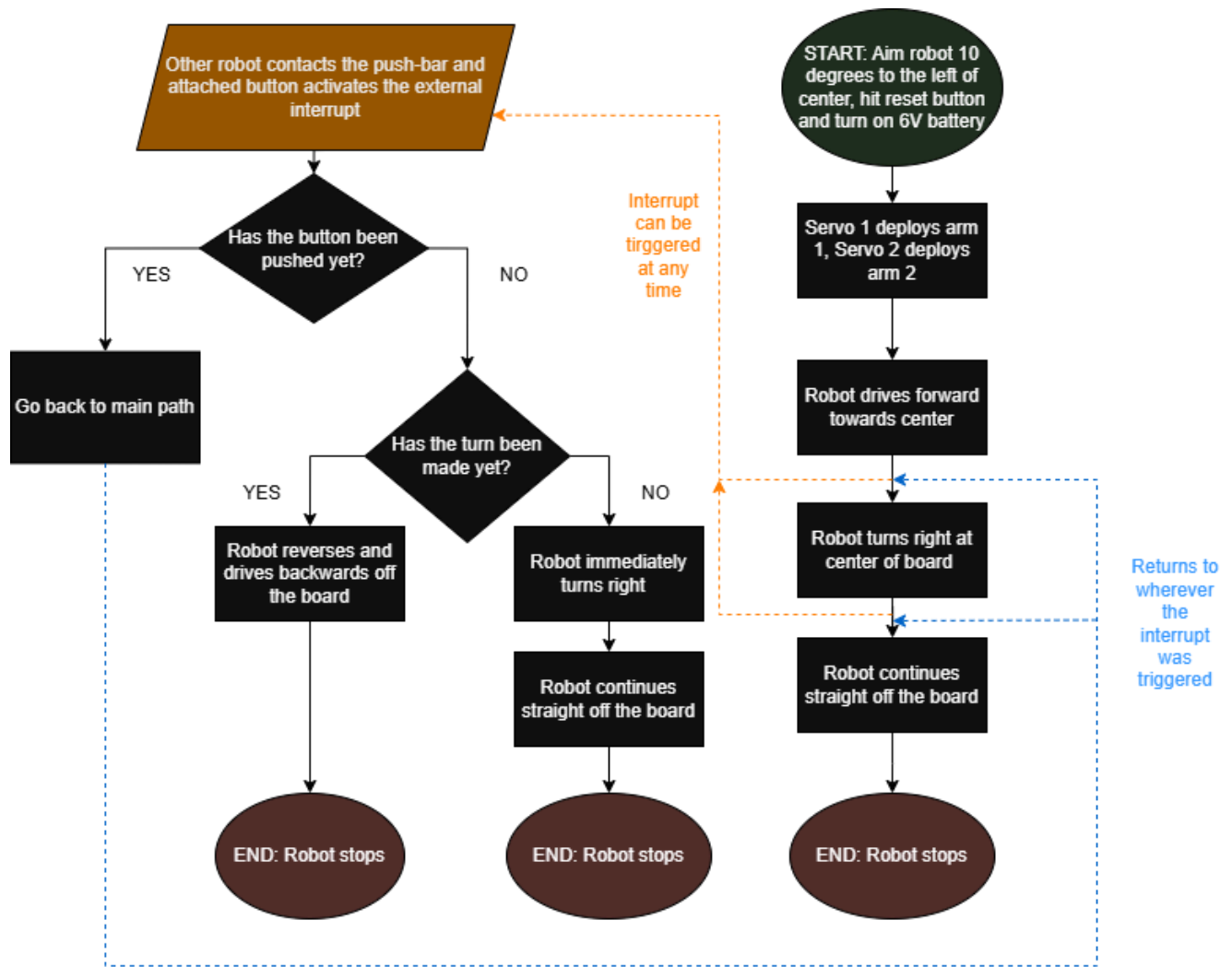


Figure 6. Flowchart of operations

9. Appendix E: Code

```

// initialize global variables
volatile int buttonpressed;
volatile int turncomplete;
int getoffboard;
volatile int backward_elapsed;
volatile int tracking;

// initialize functions
void drive_forward_delay(int t);
void drive_backward_delay(int t);

```

```

void drive_left_delay(int t);
void drive_right_delay(int t);
void delay_ms(int ms);
void stop(int t);

//ISR (Interrupt for Buttons)
// interrupt triggers when the button on INT0 is pressed
// two cases: button pressed after the turn (reverse to start), or before
the turn (exit board directly)
ISR(INT0_vect)
{
    if (buttonpressed == 0) // only respond once
    {
        tracking = 0;
        buttonpressed = 1;

        if (turncomplete == 1) // turn has already been made, reverse back to
origin
        {
            // backward_elapsed is how far forward we went, so we reverse that
plus the base distance
            int remaining = 5500 + backward_elapsed;
            drive_backward_delay(remaining);
            stop(0);
            while (1) {}
        }
        else // turn hasn't happened yet, take a direct exit
        {
            drive_right_delay(1400);
            drive_forward_delay(getoffboard);
            stop(0);
            while (1) {}
        }
    }
}

int main(void)

```

```

{
  // H-Bridge Pins
  DDRD = 0b11110000; // PD4-PD7 outputs (motors)

  // Servo Pins
  DDRB = 0b00000110; // PB2 (D10), PB1 (D9) are outputs for servo PWM

  // Interrupt Initialize, enable INTO for any logic change
  EICRA = 0b00000001;
  EIMSK = 0b00000001;
  sei(); // enable global interrupts

  // Timer1 Setup for both servos PEM
  TCCR1A = 0;
  TCCR1B = 0;

  // Fast PWM mode 14 and Non-inverting outputs for both channels
  TCCR1A |= 0b10100010; // COM1A1, COM1B1, WGM11
  TCCR1B |= 0b00011010; // WGM13, WGM12, CS11 (prescaler=8)

  // 20 ms period (50 Hz), (16MHz/8/50)-1=39999
  ICR1 = 39999;

  // Variables
  turncomplete = 0;
  buttonpressed = 0;
  getoffboard = 3000; // ms required to drive off board (hard coded)

  // SERVO 1 (Right arm)
  servo1_write(2960); // right arm down, call servo 1 function
  // SERVO 2 (Left arm)
  servo2_write(2000); // left arm down, call servo 2 function
  delay_ms(365); // deployment time, wait for it to get to position
  servo2_write(3000); // stop left arm

  // DRIVING SEQUENCE
  drive_forward_delay(1300); // drive forward to center of board
  drive_right_delay(1550); // turn right

  // Set variables

```

```

    turncomplete = 1; // turn is complete
    backward_elapsed = 0; // start tracking how far forward we go in case
ISR needs to reverse
    tracking = 1;

// DRIVING SEQUENCE CONT.
    drive_forward_delay(getoffboard);

    tracking = 0; // off the board, tracking is no longer needed
    stop(0);
    while (1) {}
}

// ----- FUNCTIONS -----
void delay_ms(int ms)
{
    for (int i = 0; i < ms; i++) {
        if (tracking == 1) backward_elapsed++; // count the ms spent moving
forward
        _delay_ms(1);
    }
}

// MOTOR CONTROL (PORTD 4-7)

void drive_forward_delay(int t)
{
    PORTD |= 0b01010000; // PINS 4 and 6 HIGH
    PORTD &= 0b01010100;

    delay_ms(t);
}

void drive_backward_delay(int t)
{
    PORTD |= 0b10100000;
    PORTD &= 0b10100100; // PINS 5 and 7 HIGH
    delay_ms(t);
}

```

```
void drive_left_delay(int t)
{
    PORTD |= 0b01100000; // PINS 5 and 6 HIGH
    PORTD &= 0b01100100;
    delay_ms(t);
}

void drive_right_delay(int t)
{
    PORTD |= 0b10010000; // PINS 4 and 7 HIGH
    PORTD &= 0b10010100;
    delay_ms(t);
}

void stop(int t)
{
    PORTD = (PORTD & 0b00001111); // Pins 4, 5, 6, 7, all LOW, motors should
stop
    delay_ms(t);
}

// SERVO (D11 + D10)
void servo1_write(int pulse) {
    OCR1A = pulse;
}

void servo2_write(int pulse) {
    OCR1B = pulse;
}
```